

DATA STRUCTURE

LAB PROGRAMS

16. a) Priority Queue Using Max Heap and Heap sort .

```
#include <stdio.h>

void heapify(int arr[], int n, int i)
{
    int largest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;
    if(left < n && arr[left] > arr[largest])
        largest = left;
    if(right < n && arr[right] > arr[largest])
        largest = right;
    if(largest != i)
    {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n)
{
    for(int i = n/2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    for(int i = n-1; i > 0; i--)
```

```

    {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;
        heapify(arr, i, 0);
    }
}

int main()
{
    int arr[] = {23, 56, 12, 34, 78, 29};
    int n = 6;
    heapSort(arr, n);
    printf("Sorted Elements: ");
    for(int i=0; i<n; i++)
        printf("%d ", arr[i]);
    return 0;
}

```

```

Output
Sorted Elements: 12 23 29 34 56 78

=== Code Execution Successful ===

```

b) Priority Queue Using Min Heap and Heap sort.

```

#include <stdio.h>

int heap[20];
int size = 0;

void insert(int value)
{
    int i = size++;

```

```

heap[i] = value;
while(i > 0)
{
    int parent = (i - 1) / 2;
    if(heap[parent] > heap[i])
    {
        int temp = heap[parent];
        heap[parent] = heap[i];
        heap[i] = temp;
        i = parent;
    }
    else
        break;
}
}

void display()
{
    for(int i = 0; i < size; i++)
        printf("%d ", heap[i]);
}

int main()
{
    int arr[] = {23, 56, 12, 34, 78, 29};
    for(int i = 0; i < 6; i++)
        insert(arr[i]);
    printf("Min Heap: ");
    display();
    return 0;
}

```

Output

Min Heap: 12 34 23 56 78 29

=== Code Execution Successful ===